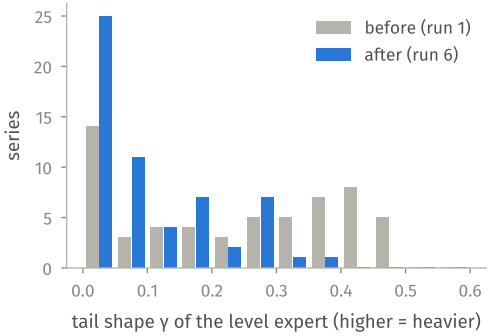
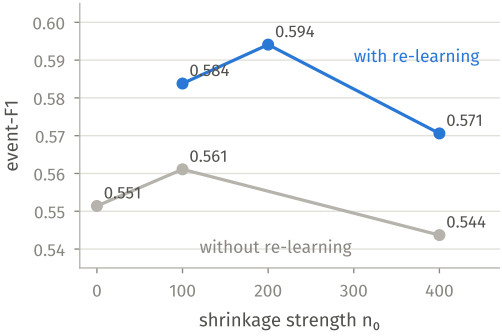
Backup: why shrink the tail shape, and by how much

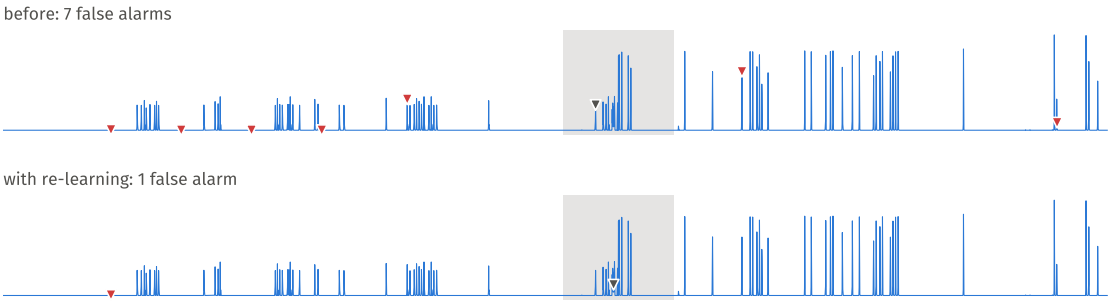




Left. Before, the level expert's fitted shape was above 0.3 on 25 of 58 series: "huge values are normal", so obvious spikes could not fire. After shrinkage most shapes sit near 0 until a series has shown enough extremes to justify a heavy tail.

Right. The strength is bracketed on both sides: 100 and 400 are worse than 200. With re-learning the best value moved from 100 to 200, because re-learning absorbs the repeat alarms that stronger shrinkage used to cause.

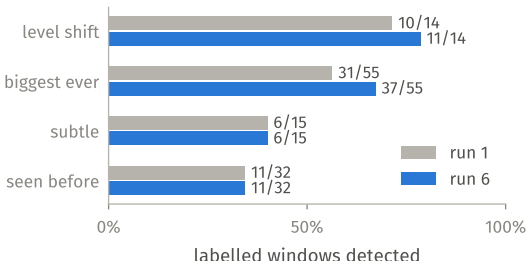
Backup: re-learning after a regime change



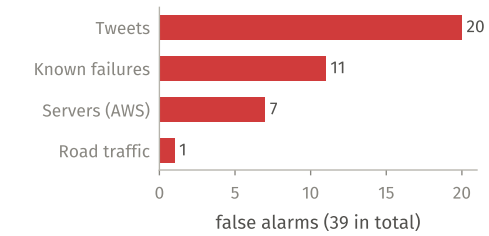
The pattern. A disk sat idle through the warm-up, then started working in bursts. The old detector re-fired the moment every 300-point cooldown ended: at points 461, 762, 1063 and 1364, exactly 301 apart.

The rule. An alarm still on at the first point after the cooldown is the same incident, so the normal has changed: rebuild every expert from the last 300 points. Result: 9 fewer false alarms, no window lost. It fired 3 times in 365,558 points, about 0.2 s each.

Backup: what we miss and where false alarms come from



Detection rate by anomaly type, before (run 1) and after (run 6).



The 39 false alarms by category.

The gains are on biggest-ever spikes (31 → 37) and level shifts (10 → 11). “Seen before” spikes did not move (11 of 32): an equally big spike already happened unlabelled, so a rarity-based detector, by design, has learned that size is normal. Most false alarms are large unlabelled bursts in tweet volume.

Backup: questions we expect

Why not deep learning?
Each signal must be learned live from a few hundred points, with no training set. Deep models need far more data per signal, adapt slowly and are hard to explain.

Is 0.594 overfitted?
Only global settings, at most 15 runs, and leave-one-category-out gives 0.594 too. The ideas were designed on all labels, so expect the hidden test a little lower.

Why is recall 0.56?
32 of 116 anomalies are “seen before” spikes. On level shifts we find 79%, on biggest-ever spikes 67%.

What does risk 10^{-5} mean?
An alarm fires only when a value is that rare for that signal. The same rule holds on every signal; no threshold is set by hand.

Is it really streaming?
Each point is processed once, and the output at time t uses data up to t only. Typical cost is 61 µs per point.

How was the 15-run budget used?
7 runs: 1 reference, 3 kept, 3 dropped. 7 more ideas were rejected from saved outputs without spending a run.

Backup: why we report event-F1, not point-adjusted F1

Point adjustment, used in many papers

Alarms raised: 1 of 8 window points



After adjustment: 8 of 8



Once any point of a labelled window fires, every point of the window counts as found. A median NAB window is 212 points long, so one lucky alarm becomes 212 true positives, while each false alarm costs only one point.

	F1 when scored by		
detector	events	point-adjusted	points
Sentinel (ours)	**0.594**	0.823	0.045
random, 1% of points	0.163	**0.919**	0.018
random, 0.28% of points	0.146	0.765	0.005
flag every point	0.945	0.168	0.168

Our final labels against detectors that know nothing. 0.28% is Sentinel's own alarm rate. Random rows: mean of 5 seeds. `python scripts/metric_check.py`, 0 runs.

Point adjustment rewards noise: random alarms on 1% of points beat Sentinel. Event-F1, the challenge metric, separates them: random 0.16, Sentinel 0.594. No metric is fool-proof: one endless alarm per series touches every window and scores 0.945 on event-F1. So we also report the alarm count (106, of which 39 false) and the delay, and keep every alarm short.